

The logo consists of a solid blue square. Inside the square, the text 'FIAP' is at the top, a vertical bar '|' is in the middle, and 'DSA' is at the bottom, all in white, bold, sans-serif font.

CHECKPOINT

# Análise Big-O e Organização de Dados

Contagem de operações, limites, logaritmos, structs e ponteiros em C

|       |  |
|-------|--|
| Nome  |  |
| RM    |  |
| Turma |  |
| Data  |  |

Modalidade: Individual | Valor: 100 pontos | Linguagem: C

Prof. Dr. Erick Toshio Yamamoto

## Orientações Gerais

### Objetivo do Checkpoint

**Demonstrar a capacidade de transformar código em uma função de custo  $T(n)$ , justificar a classe assintótica com raciocínio matemático e organizar dados utilizando struct e typedef e manipular dados por meio de ponteiros.**

- A atividade é individual e deve ser resolvida sem consulta a soluções prontas.
- Nas questões de Big-O, não será aceita apenas a classificação final: apresente o raciocínio.
- Nas questões matemáticas, mostre limites, propriedades de logaritmos e/ou estimativas por integral quando solicitado.
- Os códigos devem compilar e executar sem erros.
- Use nomes de variáveis claros e mantenha a indentação do código.
- Na parte de ponteiros, identifique claramente o significado de `&`, `*`, `->` e `NULL` e justifique o efeito das alterações na memória.

**Situação-problema** Uma empresa precisa desenvolver um pequeno sistema em C para analisar algoritmos e organizar produtos de estoque. A equipe deve justificar matematicamente a eficiência das rotinas antes de implementá-las.

### Fluxo esperado

Código  $\rightarrow T(n) \rightarrow$  Análise matemática  $\rightarrow$  Big-O  $\rightarrow$  Estrutura de dados  $\rightarrow$  Ponteiros

### Distribuição de pontos

|                                           |     |
|-------------------------------------------|-----|
| Parte 1 - Contagem exata de operações     | 15  |
| Parte 2 - Complexidade logarítmica        | 10  |
| Parte 3 - Série harmônica e integral      | 15  |
| Parte 4 - Comparação de funções           | 10  |
| Parte 5 - Struct e typedef                | 15  |
| Parte 6 - Struct aninhada                 | 10  |
| Parte 7 - Ponteiros e acesso por endereço | 25  |
| TOTAL                                     | 100 |

## Parte 1 - Contagem exata de operações

```
int soma(int vetor[], int n) {
    int soma = 0;

    for (int i = 1; i < n; i++) {
        soma += vetor[i];
    }

    return soma;
}
```

1. Quantas vezes o corpo do for é executado? [2 pts]

2. Quantas vezes a comparação  $i < n$  é realizada? [2 pts]

3. Quantas vezes ocorre  $i++$ ? [1 pt]

4. Complete a tabela de contagem de operações. [4 pts]

| Operacao         | Quantidade |
|------------------|------------|
| soma = 0         |            |
| i = 1            |            |
| i < n            |            |
| i++              |            |
| soma += vetor[i] |            |
| return soma      |            |

5. Construa a função de custo  $T(n)$ . [2 pts]

6. Determine a notacao Big-O. [2 pts]

7. Justifique por que  $O(n-1)$  e  $O(n)$  representam a mesma ordem assintotica usando o limite abaixo. [2 pts]

$$\lim_{n \rightarrow \infty} (n - 1) / n$$

## Parte 2 - Complexidade logarítmica

```
void processar(int n) {  
    int i = (n - 1) / 2;  
  
    while (i > 1) {  
        printf("%d\n", i);  
        i = i / 2;  
    }  
}
```

1. Qual é o valor inicial de  $i$  em função de  $n$ ? [1 pt]

2. Como o valor de  $i$  varia a cada iteração? [1 pt]

3. Escreva uma expressão aproximada para a quantidade de repetições. [3 pts]

*Sugestão de forma:  $\log_2((n - 1) / 2)$*

4. Simplifique a expressão usando propriedades de logaritmos. [2 pts]

5. Justifique por que a classe final pode ser escrita como  $O(\log n)$ . Não use apenas a frase "constantes são descartadas". [3 pts]

### Lembrete matemático

$\log_b(a/c) = \log_b(a) - \log_b(c)$ . A base do logaritmo não altera a classe Big-O, pois a mudança de base introduz apenas um fator constante.

## Parte 3 - Série harmônica e estimativa por integral

```
long long algoritmo(int n) {  
    long long contador = 0;  
  
    for (int i = 1; i <= n; i++) {  
        for (int j = i; j <= n; j += i) {  
            contador++;  
        }  
    }  
  
    return contador;  
}
```

1. Para  $i = 1$ , aproximadamente quantas vezes o laço interno executa? [1 pt]

2. Para  $i = 2$ ? Para  $i = 3$ ? Generalize para um valor  $i$  qualquer. [2 pts]

3. Escreva  $T(n)$  na forma de soma. [3 pts]

$$T(n) \sim n + n/2 + n/3 + \dots + n/n$$

4. Coloque  $n$  em evidencia e identifique a série matemática resultante. [2 pts]

5. Use a integral de  $1/x$  para justificar o crescimento da série harmônica. [3 pts]

$$\text{integral de 1 até n de } (1/x) dx = \ln(n)$$

6. Determine a complexidade Big-O do algoritmo e justifique. [2 pts]

7. Explique por que dois for aninhados, neste caso, não implicam  $O(n^2)$ . [2 pts]

### Ponto-chave

A quantidade de repetições do laço interno depende de  $i$ . Portanto, não basta multiplicar  $n$  por  $n$  sem analisar o comportamento real das iterações.

## Parte 4 - Comparação de funções

Determine a classe Big-O de cada função. Em pelo menos tres casos, justifique usando um limite do tipo  $T(n)/g(n)$ . [6 pts]

| T(n)                | Big-O | Justificativa curta |
|---------------------|-------|---------------------|
| $n - 1$             |       |                     |
| $n + 1$             |       |                     |
| $4n + 10$           |       |                     |
| $\log_2(n - 1) - 1$ |       |                     |
| $n \log n + 5n$     |       |                     |
| $(n^2 + n)/2$       |       |                     |
| $3n^2 + 7n + 20$    |       |                     |
| $2^n + n^3$         |       |                     |

### Interpretação dos limites

| Resultado do limite       | Interpretação                             |
|---------------------------|-------------------------------------------|
| $0 < L < \text{infinito}$ | mesma ordem de crescimento                |
| $L = 0$                   | $T(n)$ cresce mais lentamente que $g(n)$  |
| $L = \text{infinito}$     | $T(n)$ cresce mais rapidamente que $g(n)$ |

### Questão de reflexao

Ordene da menor para a maior taxa de crescimento:  $O(n^2)$ ,  $O(1)$ ,  $O(n \log n)$ ,  $O(\log n)$ ,  $O(2^n)$ ,  $O(n)$ . [2 pts]

---



---

Explique por que Big-O analisa o comportamento quando  $n$  cresce, e não apenas o tempo medido em uma unica execucao. [2 pts]

---



---



---

## Parte 5 - Organização de dados com struct e typedef

A empresa agora precisa representar produtos de estoque. Implemente um programa completo em C utilizando obrigatoriamente typedef struct.

```
typedef struct {  
    int id;  
    char nome[50];  
    float preco;  
    int quantidade;  
} Produto;
```

1. Leia pelo teclado os dados de um produto.
2. Armazene os valores na struct Produto.
3. Calcule o valor total em estoque:  $\text{valor} = \text{preco} * \text{quantidade}$ .
4. Exiba todos os dados de forma organizada.
5. Acesse os campos utilizando o operador ponto (.).

**Cole abaixo o trecho principal do seu código ou descreva a logica implementada. [10 pts]**

---

---

---

---

---

---

---

---

Explique por que typedef melhora a legibilidade e a manutenção do código quando trabalhamos com structs. [5 pts]

---

---

---

## Parte 6 - Struct aninhada

Amplie o sistema criando uma estrutura Data e adicionando a validade ao Produto.

```
typedef struct {  
    int dia;  
    int mes;  
    int ano;  
} Data;  
  
typedef struct {  
    int id;  
    char nome[50];  
    float preco;  
    int quantidade;  
    Data validade;  
} Produto;
```

1. Cadastre e exiba a data de validade utilizando acesso encadeado, por exemplo produto.validade.dia. [5 pts]

---

---

---

2. Explique por que uma struct aninhada melhora a organização dos dados. [5 pts]

---

---

---

## Parte 7 - Ponteiros e acesso por endereço

Nesta parte, analise como um ponteiro armazena endereços de memória, como a desreferência altera o dado original e como ponteiros são utilizados com structs. Não utilize malloc/free nesta atividade.

### 7.1 - Rastreamento de memória [8 pts]

```
#include <stdio.h>

int main(void) {
    int valor = 10;
    int *ptr = &valor;

    printf("%d\n", valor);
    printf("%p\n", (void*)ptr);
    printf("%d\n", *ptr);

    *ptr = 25;
    printf("%d\n", valor);

    return 0;
}
```

a) Explique o que está armazenado em ptr. [2 pts]

---

b) Explique a diferença entre ptr, \*ptr e &valor. [3 pts]

---

---

c) Qual será o valor de valor após a instrução \*ptr = 25? Justifique por que a variável original foi alterada. [2 pts]

---

d) Faça um pequeno diagrama contendo: variável, endereço, ponteiro e valor apontado. [1 pt]

---

### 7.2 - Função recebendo um ponteiro [7 pts]

```
void dobrar(int *numero) {
    // COMPLETE A FUNCAO
}

int main(void) {
    int x = 10;
    dobrar(&x);
    printf("%d\n", x);
    return 0;
}
```

a) Complete a função dobrar para que x passe a valer 20, sem retornar um novo valor. [3 pts]

---

---

b) Explique por que a chamada utiliza dobrar(&x) em vez de dobrar(x). [2 pts]

---

---

c) Compare passagem por valor e passagem por endereço neste exemplo. [2 pts]

---

---

### 7.3 - Ponteiro para struct e operador -> [7 pts]

```
typedef struct {
    int id;
    char nome[50];
    float preco;
} Produto;

void aplicarDesconto(Produto *p, float desconto) {
    // COMPLETE UTILIZANDO ->
}

int main(void) {
    Produto prod = {1, "Teclado", 200.0};
    aplicarDesconto(&prod, 10.0);
    printf("%.2f\n", prod.preco);
    return 0;
}
```

a) Implemente aplicarDesconto para alterar o preço original utilizando p->preco. [4 pts]

---

---

b) Explique a equivalência entre p->preco e (\*p).preco. [2 pts]

---

---

c) Por que a main passa &prod para a função? [1 pt]

---

---

### 7.4 - Segurança com ponteiros [3 pts]

```
Produto *p = NULL;

if (p != NULL) {
    printf("%.2f\n", p->preco);
}
```

Explique por que é uma boa prática inicializar um ponteiro com NULL quando ele ainda não possui um endereço válido e por que devemos verificar p != NULL antes de usar \*p ou ->. [3 pts]

---

---

---

## Entrega e Checklist Final

**Formato de entrega**

Envie um unico arquivo compactado contendo o PDF com as respostas e os códigos-fonte .c solicitados. O nome do arquivo deve seguir o padrao: checkpoint\_RM.zip.

- questões.pdf - desenvolvimento das análises e justificativas matemáticas;
- bigO.c - códigos utilizados nas questões de análise;
- produto.c - implementacao com struct, typedef e struct aninhada.
- ponteiros.c - implementação das questões da Parte 7.

**Checklist antes de enviar**

- ☐ Todas as questões possuem justificativa, e não apenas a resposta final.
- ☐ Os limites solicitados foram desenvolvidos.
- ☐ A série harmônica foi relacionada a integral de  $1/x$ .
- ☐ Os códigos compilam sem erros.
- ☐ A struct Produto utiliza typedef.
- ☐ A validade foi implementada como struct aninhada.
- ☐ As questões de ponteiros identificam corretamente  $\&$ ,  $*$ ,  $\rightarrow$  e NULL.
- ☐ A função com ponteiro altera a variável original de forma correta.
- ☐ O ponteiro para struct utiliza o operador  $\rightarrow$  corretamente.
- ☐ O arquivo final esta identificado com nome e RM.

**Boa atividade!**